

ECSE 343: Numerical Methods for Engineers- Assignment 1- part b

Due Date: 22nd Feb. 2024

Student Name: Karen Cherry Chen Lai

Student ID: 261107674

Please type your answers and write you code in this .mlx script. If you choose to provide the handwritten answers, please scan your answers and include those in SINGLE pdf file.

Please submit this .mlx file along with the PDF copy of this file.

Table of Contents

Please type your answers and write you code in this .mlx script. If you choose to provide the handwritten answers, please scan your answers and include those in SINGLE pdf file.....	1
Please submit this .mlx file along with the PDF copy of this file.....	1
Question 1: Cholesky Least Squares Estimation (10 marks).....	1
Question 2: Proofs (10 marks).....	3
Question 3: QR Decomposition (10 marks).....	5
Question 4: Polynomial Fitting using Least Squares Estimation (10 marks).....	6
Appendix	12
Implement the Cholesky Decompositon fucntion here.....	12
Implement Least Square Solver for Cholesky here.....	13
Implement Polynomial Matrix here	14
Implement QR Methods here.....	14
Function to generate Hilbert Matrix is provided here.....	15

Question 1: Cholesky Least Squares Estimation (10 marks)

(a) **Cholesky factorization** is an alternative to LU decomposition which can be applied on symmetric positive definite matrices. A symmetric positive definite matrix M must satisfy following two conditions:

1. The matrix M must be symmetric, i.e. $M = M^T$
2. The matrix M must be positive definite, i.e. $x^T M x > 0$ for all $x \in \mathbb{R}^{N \times 1}$, $\|x\| \neq 0$.

If the above two conditons are met then the matrix M can be factored as

$$M = U^T U$$

where U is a $N \times N$ the lower upper matrix with real and positive diagonal enteries. The enteries of matrix U are given by the following expression

$$U_{i,j} = \begin{cases} \sqrt{\mathbf{M}_{i,i} - u_i(1:i-1)^T u_i(1:i-1)}, & \text{if } i = j \\ \frac{1}{U_{i,i}} (\mathbf{M}_{i,j} - u_i(1:i-1)^T u_j(1:i-1)), & \text{if } i < j \\ 0, & \text{otherwise} \end{cases}$$

The cost of Cholesky factorization algorithm is roughly half than the LU decomposition. Your task is to implement the Cholesky decomposition algorithm, the use the outline of the function named *CholeskyDecomposition* provided in the appendix.

Use the cell below to test your Cholesky decomposition code by computing the Frobenius norm of $U^T U - M$.

```
M= [2 -1 0; -1 2 -1; 0 -1 2];
U = CholeskyDecompositon(M);
Error = norm(U'*U-M);
disp(Error);
```

NaN

b) In regression problems we often need to solve the normal equations, $A^T A x = A^T b$. Show that the matrix $(A^T A)$ is symmetric positive definite, and thus can be factored using Cholesky Decomposition.

% ANSWER

b) Given two orthonormal matrices U and Q , show that product of these matrices, UQ , is also orthonormal.

% ANSWER

c) If A is an invertible matrix and Q is an orthonormal matrix, show that $\kappa(QA) = \kappa(A)$.

% ANSWER

Question 3: QR Decomposition (10 marks)

In this question we will develop the different algorithms to implement QR factorization.

a) Use the cell below to implement a function that computes Gram-Schmidt based QR decomposition of a input matrix A.

Use the cell below to test the your method by comparing the computing the $\|A - QR\|_F$. Use the Frobenius norm in the MATLAB, comment on your result.

```
A = rand(15,6); % A is a 15x6 matrix of random numbers
[Qgs,Rgs] = gschmidt(A);

%use the norm to check
normGS = norm(A-Qgs*Rgs,'fro') ;

disp(normGS);
```

4.0981e-16

Comment: The Gram-Schmidt based QR decomposition has potential unstable numerical issues, particularly for large matrices and matrices that have nearly linearly dependent columns, due to the rounding errors present in the vectors subtraction from projections. It is not recommended to use the GS based because it is not convenient for matrices close to singular or ill-conditioned.

b) Use the cell below to implement a function that uses the Modified Gram-Schmidt approach to compute the QR decomposition of the input matrix A.

Use the cell below to test the your method by comparing the computing the $\|A - QR\|_F$. Use the Frobenius norm in the MATLAB, comment on your result.

```
A = rand(15,6); % A is a 15x6 matrix of random numbers
[Qmgs,Rmgs] = mgschmidt(A);

normMGS = norm(A-Qmgs*Rmgs,'fro');
```

```
disp(normMGS);
```

3.8957e-16

Comment: The modified Gram-Schmidt based QR decomposition is a better choice compared to the one of GS based since it is much more stable because it avoids the errors accumulation through changing the order of vector computation in each iteration of projections. Hence modified GS approach provides much better numerical stability.

Question 4: Polynomial Fitting using Least Squares Estimation (10 marks)

Polynomial fitting is one of the many applications of Least-Squares Approximation. Given the experimental data, shown in Table below,

$$\begin{array}{l} \text{Input, } x | \quad x_0 \quad x_1 \quad x_2 \quad x_3 \quad \dots \quad x_{m-1} \quad x_m \\ \text{Output, } y | \quad y_0 \quad y_1 \quad y_2 \quad y_3 \quad \dots \quad y_{m-1} \quad y_m \end{array}$$

we can fit a n^{th} degree polynomial, $p(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \dots + a_nx^n$. To find the coefficients, $\mathbf{a} = [a_0, a_1, a_2, \dots, a_n]$, we can solve the linear system shown below.

$$\underbrace{\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m-1} & x_{m-1}^2 & \dots & x_{m-1}^n \\ 1 & x_m & x_m^2 & \dots & x_m^n \end{bmatrix}}_{\mathbf{M}} \underbrace{\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}}_{\mathbf{a}} = \underbrace{\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{m-1} \\ y_m \end{bmatrix}}_{\mathbf{Y}}$$

The system $\mathbf{M} \mathbf{a} = \mathbf{Y}$ can then be solved to compute the coefficients in vector $\mathbf{a}$.

a) Your first task is to write a function that takes the input data vector x and degree of the polynomial, then returns the polynomial matrix $\mathbf{M}$. Complete the function *PolynomialMatrix* in the appendix.

```
% test your code here by inputing X= [ 1 2 3] for degree 3.
```

```
% You can manually verify if this matrix is implemented correctly.
```

```
X= [ 1; 2; 3];  
Mtest = PolynomialMatrix(X,3)
```

```
Mtest = 3x4  
     1     1     1     1  
     1     2     4     8  
     1     3     9    27
```

b) The least squares method can be used to solve the overdetermined linear system $\mathbf{M} \mathbf{a} = \mathbf{Y}$ to obtain the polynomial coefficients. This can be done by directly solving the Normal equations (using cholesky decomposition for example) or by using QR decomposition. In the code below, use the **MATLAB's QR decomposition function** to find the polynomial coefficients if we assume a polynomial model of order 1.

```
% Load the Input Data from the provided .mat file

load('/Users/karenchen/Desktop/ECSE 343/Assignments/Assignment 2/Polynomial_Fitting_Data.mat');

x = x_given;
y = y_given;
deg = 1;

% get Polynomial Matrix
M = PolynomialMatrix(x,deg);

% MATLAB QR Decomposition
[Q,R] = qr(M,0);

% Find the polynomial coefficients using Q & R matrices obtained above.
% For forward and backward substitution functions use the forward
% substitution and the backward substitution you wrote in Assignment 1

% Name the Coefficients obtained QR as Ar

% write your code here
% R_upper = R(1:deg+1, 1:deg+1); % Upper square matrix of R
% Q_transpose_y = Q' * y;
% b = Q_transpose_y(1:deg+1); % Corresponding upper vector of Q'*y
% coefficients = R_upper \ b;
% Ar = coefficients;
b = Q'*y;
Ar = backward_sub(R,b);
```

Now that you have obtained the coefficients for the first order polynomial model, $p(x) = a_0 + a_1x$, we can test this model by evaluating $p(x)$ at different values of x between 0 to 7. **Run the code below and explain the results obtained in the figure. Is this order sufficient?**

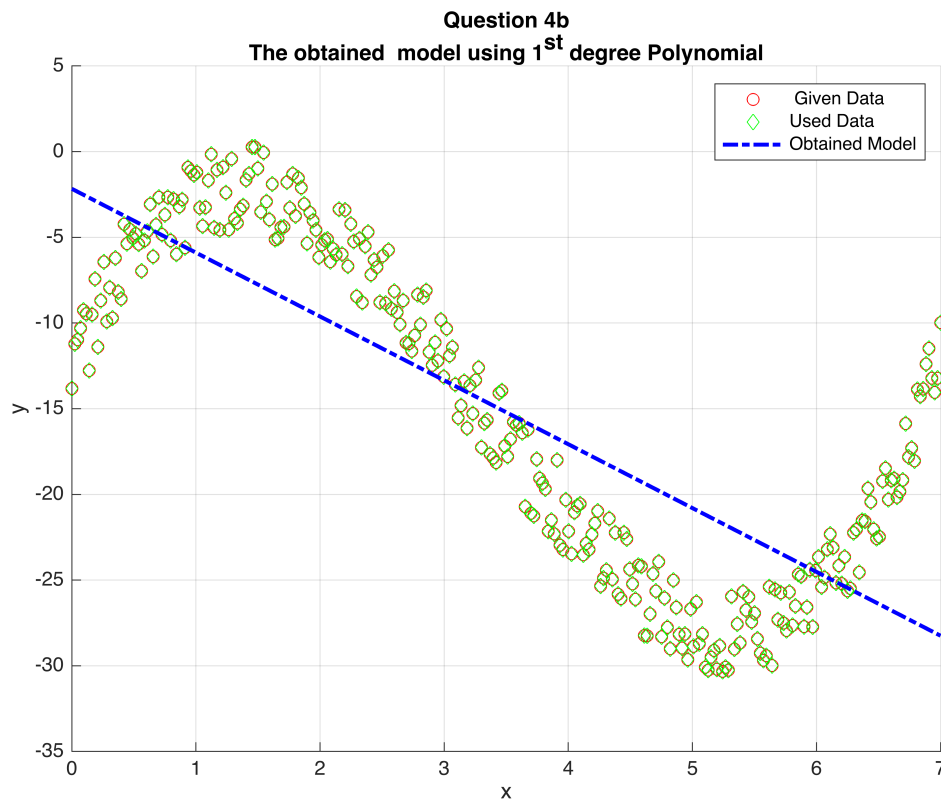
```
xtest = x;

px = zeros(length(xtest),1);
powers = 0:deg;
```

```

for I = 1:length(powers)
    px = px + Ar(I)*xtest.^powers(I);
end
%plots
figure;
hold on;
plot(x,y,'ro');
plot(x,y,'gd');
plot(xtest,px,'b-.','Linewidth',2);
xlabel('x');
ylabel('y');
legend(' Given Data','Used Data','Obtained Model');
grid on;
title({'Question 4b';'The obtained model using 1st degree Polynomial'});

```



The graph above displays a curving trend with no outliers. the obtained model line does not well-fit to data points trend. As this graph illustrates, it does not have a straight line for the data points but instead it has a varying trend. Therefore, this order is not sufficient since the obtained model line does not fit well to the data points.

c) In this part we will use a higher order model (a polynomial of order 6), but only a limited amount of input data points to generate this model (we will use only **15 data points**). Use QR decomposition function to find the polynomial coefficients for the **6th degree polynomial**.

```

% Load the Input Data from the provided .mat file

```



```
load('/Users/karenchen/Desktop/ECSE 343/Assignments/Assignment 2/Polynomial_Fitting_Data.mat');
```

```
deg = 6; %polynomial degree
x = x_given(1:20:end); % x has 15 data points!!
y = y_given(1:20:end); % y has 15 data points!!
```

```
% get Polynomial Matrix
Mp= PolynomialMatrix(x,deg);
```

```
% MATLAB QR Decomposition
[Q,R] = qr(Mp,0);
```

```
disp(R);
```

```
1.0e+04 *
```

```
-0.0004    -0.0013    -0.0057    -0.0292    -0.1585    -0.8948    -5.1946
         0     0.0008     0.0051     0.0310     0.1856     1.1181     6.7959
         0         0     0.0014     0.0138     0.1054     0.7350     4.9361
         0         0         0    -0.0025    -0.0322    -0.2966    -2.3716
         0         0         0         0    -0.0042    -0.0687    -0.7425
         0         0         0         0         0     0.0070     0.1371
         0         0         0         0         0         0     0.0113
```

```
%-----
% Find the polynomial coefficients using Q & R matrices obtained above.
```

```
% Name the Coefficeints obtained QR as Ar
```

```
% write your code here
b = Q' * y;
Ar = backward_sub(R,b);
```

We can test this model by evaluating $p(x) = a_0 + a_1x + \dots + a_6x^6$ at different values of x between 0 to 7. **Is this a good model? Comment on the results obtained in figure Question 5c, and explain the cause of what you see.**

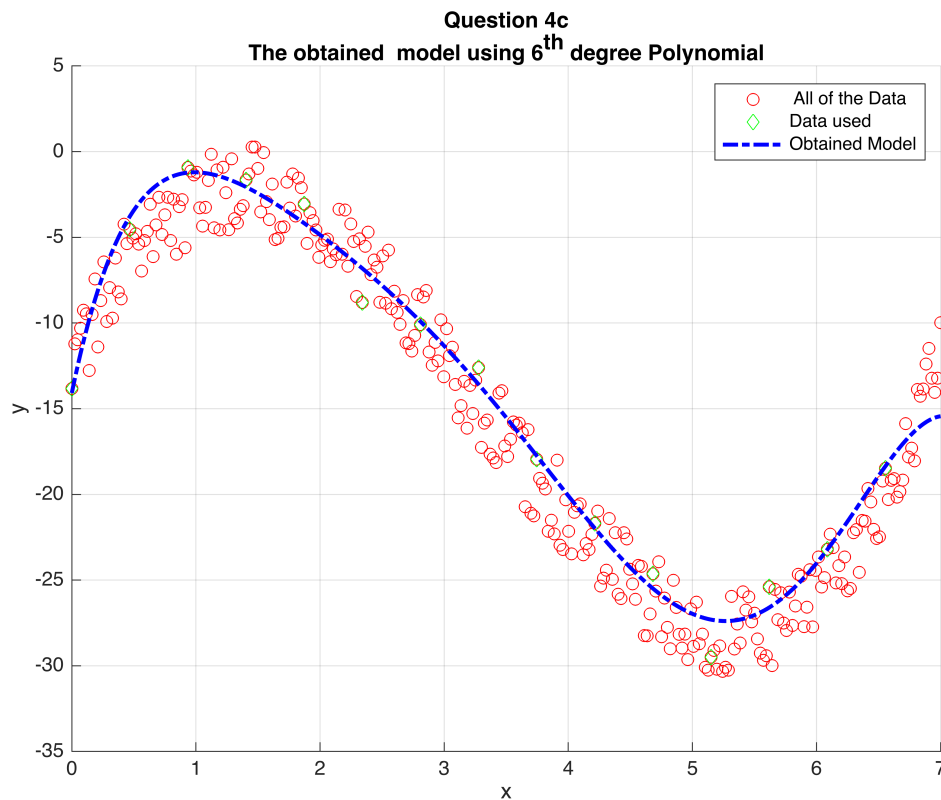
```
xtest = linspace(0,7,701);
xtest = xtest';
powers = 0:deg;
px = zeros(length(xtest),1);

for I = 1:length(powers)
    px = px + Ar(I)*xtest.^powers(I);
end
```

```

%plots
figure
hold on
plot(x_given,y_given,'ro') % plot all Data
plot(x,y,'gd') % plot Used Data
plot(xtest,px,'b-.','Linewidth',2)
xlabel('x')
ylabel('y')
legend(' All of the Data','Data used','Obtained Model')
title({'Question 4c';'The obtained model using 6th degree Polynomial'})
grid on

```



The plot above demonstrates a well-fit obtained model line to the data points. Hence it is a good model. However, high degree polynomials could lead to overfitting as we see in this plot, which is not advantageous for generalizing new data points since it fits the data points too close. The reason why I see this is because of high degree polynomials as I mentioned before, particularly, for a small subset of data points like this situation since it lacks generalization to other datasets.

d) In this part we will use a polynomial of order **14**, again this will be used with limited amount of data (we will use only **15 data points, same as part c**). Use QR decomposition function to find the polynomial coefficients for the **14th degree polynomial**.

```

% Load the Input Data from the provided .mat file

```

```

load('/Users/karenchen/Desktop/ECSE 343/Assignments/Assignment 2/Polynomial_Fitting_Da

```

```
deg =14
```

```
deg = 14
```

```
x = x_given(1:20:end); % x has 15 data points!!
y = y_given(1:20:end); % y has 15 data points!!

% get Polynomial Matrix
Mp= PolynomialMatrix(x,deg);

% MATLAB QR Decomposition
[Q,R] = qr(Mp);

%-----
% Find the polynomial coefficients using Q & R matrices obtained above.

% Name the Coefficeints obtained QR as Ar

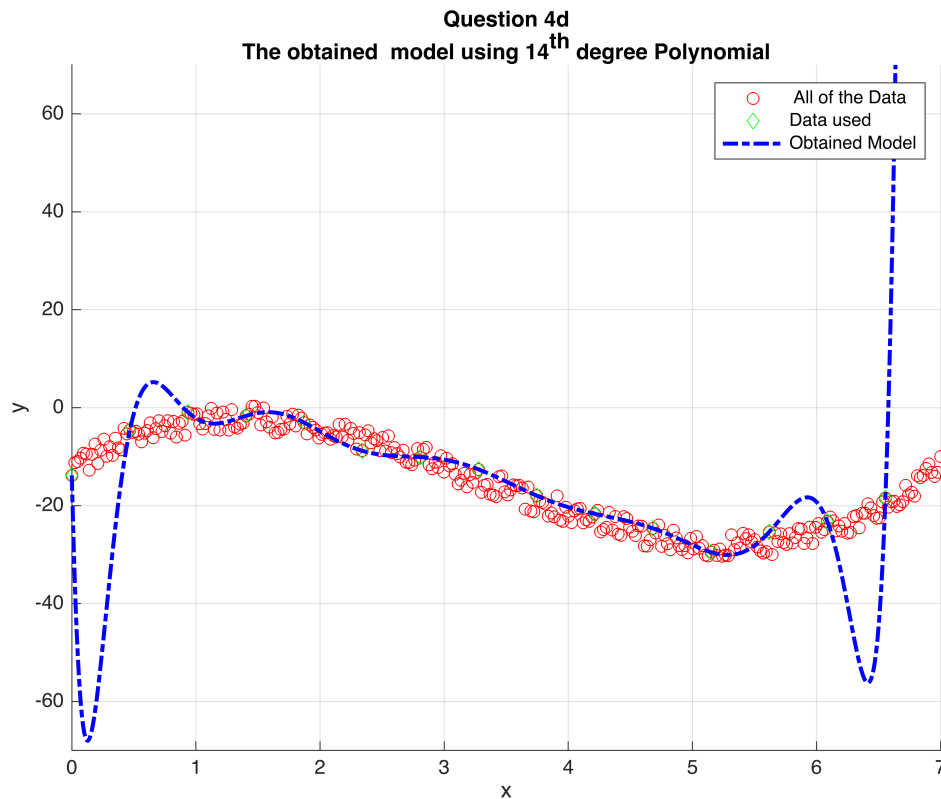
% write your code here
b = Q' * y;
Ar = backward_sub(R,b);
```

We can test this model by evaluating $p(x)$ at different values of x between 0 to 7. **Is this a good model? Comment on the results obtained in figure Question 5d, and explain the cause of what you see.**

```
xtest = linspace(0,7,701);
xtest = xtest';
powers = 0:deg;
px = zeros(length(xtest),1);

for I = 1:length(powers)
    px = px + Ar(I)*xtest.^powers(I);
end
%plots
figure('Name','Question 4d','NumberTitle','off')
hold on
plot(x_given,y_given,'ro') % plot all Data
plot(x,y,'gd') % plot Used Data
plot(xtest,px,'b-.','Linewidth',2)
xlabel('x')
ylabel('y')
ylim([-70 70])
legend(' All of the Data','Data used','Obtained Model')
```

```
title({'Question 4d'; 'The obtained model using 14th degree Polynomial'})
grid on
```



This is not a good model. The plot above displays irregular trends at the beginning and at the end, close to 0 and 6.5 respectively. The reason why this phenomenon happens is that the 14th degree polynomial tries to covers all the data points which causes it overfits the data. In other words, it highlights the fluctuations among the data sets instead of just underlying the trend. Hence it is a bad model.

Appendix

Implement the Cholesky Decompositon fucntion here.

```
function U = CholeskyDecompositon(M)
if(size(M,1) ~= size(M,2))
    disp('Matrix M must be square!');

end
U = zeros(size(M));
% Write your code here.
[n,n] = size(U);

for i = 1:n
    for j = 1+i:n
```

```

        if i==j
            U(i,j) = sqrt(M(i,i) - U(1:i-1, i)' * U(1:i-1,i) );

        elseif j>i
            U(i,j) = (M(i,j) - U(1:i-1, i)' * U(1:i-1,j) ) / U(i,i);

        else
            U(i,j) = 0;
        end
    end
end
end
end

```

Implement Least Square Solver for Cholesky here

```

function x = LeastSquareSolver_Cholesky(A,b)
% get M = A^T A
M = A'*A;

% write your code here
% Step 1. Call the choesky function written in part (a) to get Lower triangular matrix
% M = L L^T
U = CholeskyDecompositon(M);

% Step 2. Use the forward substitution function named ForwardSubs() you wrote in Ass
y = forward_sub(U', A' * b);

% Step 3. Use the backward substitution function BackwardSubs() you wrote in Assignm
x = backward_sub(U,y);

end

% forward and backward substitution functions from Assignment 1
function y = forward_sub(L,b)
% y is the vector
% L is the lower triangular matrix
n = length(b);
y = zeros(n,1);
for i = 1:n
    y(i) = ( b(i) - L(i, 1: i-1) * y(1:i-1) ) / L(i,i);
end
end

function x = backward_sub(R, b)
% X is the vector
% U is the upper triangular matrix
n = length(b);
x = zeros(n, 1); % Preallocate space for the solution vector x

```

```

    for i = n:-1:1
        x(i) = (b(i) - R(i, i+1:n) * x(i+1:n)) / R(i, i);
    end
end

```

Implement Polynomial Matrix here

```

function M = PolynomialMatrix(x,n)
% write your code here
% n is the degree of the polynomla
% x is the vector of input data
M = ones(length(x), n+1);

    for i = 1:n
        M(:, i+1) = x.^i;
    end
end

```

Implement QR Methods here

- Classical GramSchmidt

```

function [Q,R] = gschmidt(A)
% A is the input matrix-
% A could be a overdetermined matrix or a sqaure matrix
% Q is orthonormal matrix
% R is the upper trianlgular matrix

% Write your code here.
[x,y] = size(A);

Q = zeros(x,y);
R = zeros(y,y);

    for j = 1:y
        z = A(:,j);
        for i = 1:j-1
            R(i,j) = Q(:, i)' * A(:,j);
            z = z - R(i,j) * Q(:,i);
        end

        R(j,j) = norm(z);
        Q(:,j) = z/ R(j,j);
    end
end

```

- Modified Gram-Schmidt

```
function [Q,R] = mgschmidt(A)
% A is the input matrix-
% A could be a overdetermined matrix or a square matrix
% Q is orthonormal matrix
% R is the upper triangular matrix

% Write your code here.
[x,y] = size(A);

Q = zeros(x,y);
R = zeros(y,y);

for i = 1:y
    R(i,i) = norm(A(:,i));
    Q(:,i) = A(:,i) / R(i,i);
    for j = i+1:y
        R(i,j) = Q(:,i)' * A(:,j);
        A(:,j) = A(:,j) - R(i,j) * Q(:, i);
    end
end
end
```

Function to generate Hilbert Matrix is provided here

```
function A = Hilbert_matrix(size)

A = zeros(size,size);

for i=1:size
    for j=1:size
        A(i,j) = 1/(i+j-1);
    end
end
end
```